

Лабораторная работа №1

Шигапов Камиль Ильясович

Группа 4404

Задание 1.

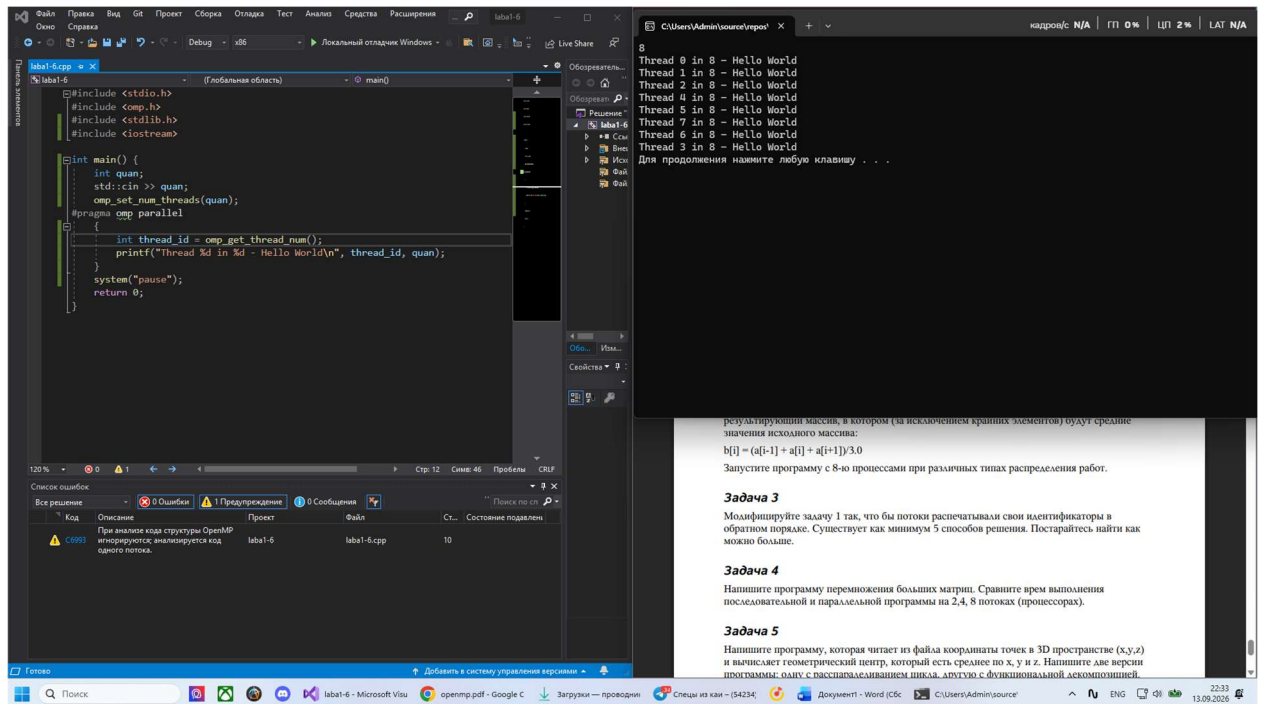


Рис. 1. Код программы и её выполнение

В вышеприведенном рисунке, с консоли запущено программа для 8 потоков. Вывод неидентичен, ниже представлено несколько запусков одной и той же программы:

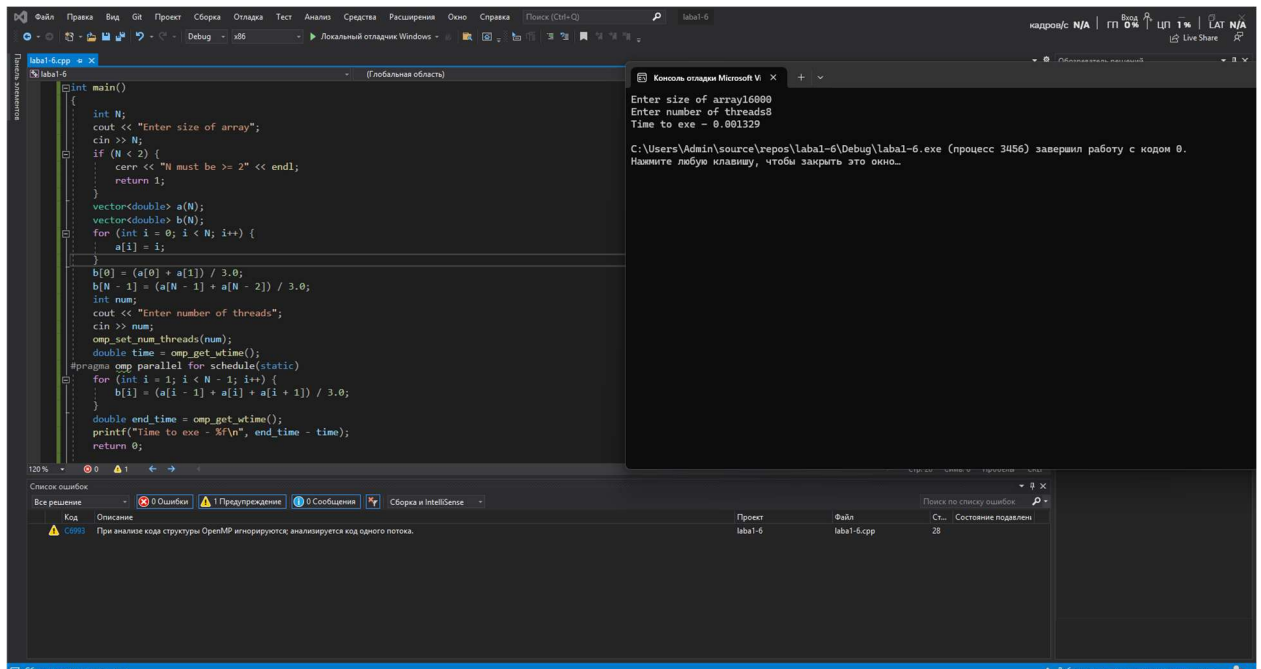
```
C:\Users\Admin\source\repos\ X + v
8
Thread 0 in 8 - Hello World
Thread 1 in 8 - Hello World
Thread 2 in 8 - Hello World
Thread 4 in 8 - Hello World
Thread 3 in 8 - Hello World
Thread 5 in 8 - Hello World
Thread 6 in 8 - Hello World
Thread 7 in 8 - Hello World
Для продолжения нажмите любую клавишу . . . |
```

```
C:\Users\Admin\source\repos\ X + v
8
Thread 0 in 8 - Hello World
Thread 4 in 8 - Hello World
Thread 5 in 8 - Hello World
Thread 2 in 8 - Hello World
Thread 1 in 8 - Hello World
Thread 6 in 8 - Hello World
Thread 3 in 8 - Hello World
Thread 7 in 8 - Hello World
Для продолжения нажмите любую клавишу . . . |
```

```
C:\Users\Admin\source\repos\ x + -  
8  
Thread 0 in 8 - Hello World  
Thread 1 in 8 - Hello World  
Thread 2 in 8 - Hello World  
Thread 4 in 8 - Hello World  
Thread 5 in 8 - Hello World  
Thread 3 in 8 - Hello World  
Thread 7 in 8 - Hello World  
Thread 6 in 8 - Hello World  
Для продолжения нажмите любую клавишу . . . |
```

Вывод неидентичен, потому что потоки выполняются одновременно и вне зависимости друг от друга, никто их не синхронизирует (нет критических секций, барьеров, атомарных операций и блокировок).

Задание 2.



```
int main()  
{  
    int N;  
    cout << "Enter size of array";  
    cin >> N;  
    if (N < 2) {  
        cerr << "N must be >= 2" << endl;  
        return 1;  
    }  
    vector<double> a(N);  
    vector<double> b(N);  
    for (int i = 0; i < N; i++) {  
        a[i] = i;  
    }  
    b[0] = (a[0] + a[1]) / 3.0;  
    b[N - 1] = (a[N - 1] + a[N - 2]) / 3.0;  
    int num;  
    cout << "Enter number of threads";  
    cin >> num;  
    omp_set_num_threads(num);  
    double time = omp_get_wtime();  
    #pragma omp parallel for schedule(static)  
    for (int i = 1; i < N - 1; i++) {  
        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;  
    }  
    double end_time = omp_get_wtime();  
    printf("Time to exe - %f\n", end_time - time);  
    return 0;  
}
```

Консоль отладки Microsoft VI

```
Enter size of array16000  
Enter number of threads8  
Time to exe - 0.001329  
C:\Users\Admin\source\repos\labal-6\Debug\labal-6.exe (процесс 3456) завершил работу с кодом 0.  
Нажмите любую клавишу, чтобы закрыть это окно...
```

Список ошибок

Код	Описание	Проект	Файл	Стр.	Состояние подделен
C4993	При анализе кода структуры OpenMP игнорируются анализируется код одного потока.	labal-6	labal-6.cpp	28	

В данном коде используется `#pragma omp parallel for schedule(static)`, а крайние элементы `b[0]` и `b[N-1]` вычисляются последовательно

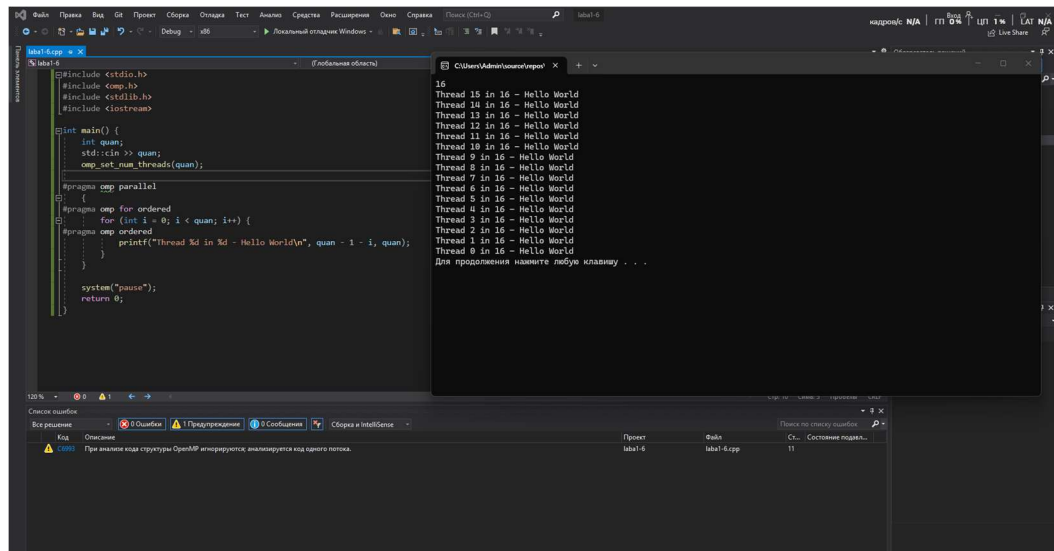
Программа корректно распараллеливает цикл, время выполнения зависит от `N` и количества потоков.

Задание 3.

Способ 1. Директива ordered

Идея: использовать конструкцию `#pragma omp for ordered` вместе с `#pragma omp ordered`. OpenMP гарантирует, что блок `ordered` выполняется в порядке итераций цикла. Так как цикл идёт от 0 до `quan-1`, для обратного порядка выводим `quan - 1 - i`

Простота, надежность, минимальные накладные расходы



```
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>
#include <iostream>

int main() {
    int quan;
    std::cin >> quan;
    omp_set_num_threads(quan);

    #pragma omp parallel
    {
        #pragma omp for ordered
        for (int i = 0; i < quan; i++) {
            #pragma omp ordered
            printf("Thread %d in %d - Hello World\n", quan - 1 - i, quan);
        }

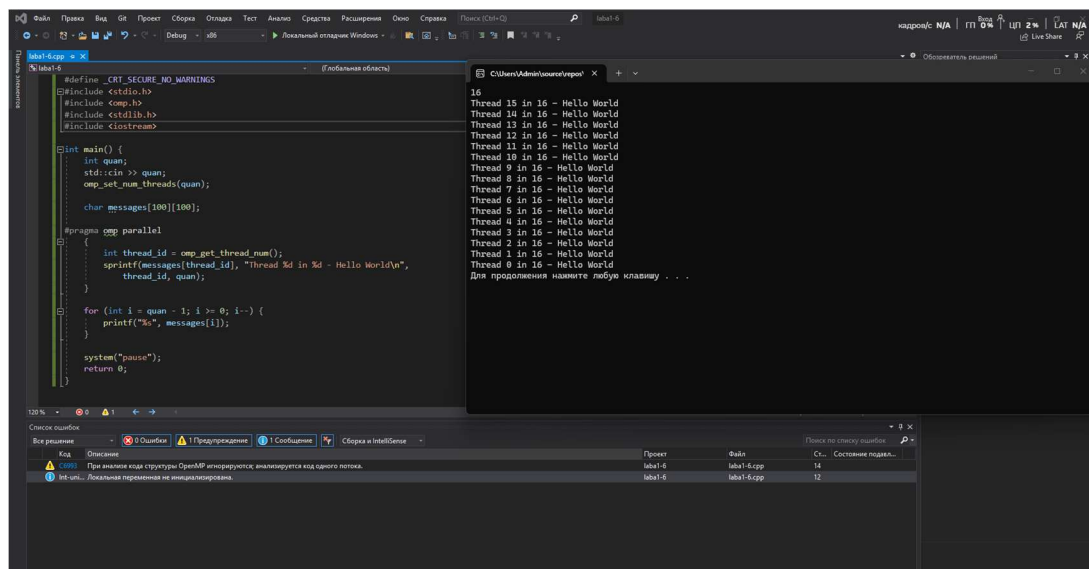
        system("pause");
        return 0;
    }
}
```

16
Thread 15 in 16 - Hello World
Thread 14 in 16 - Hello World
Thread 13 in 16 - Hello World
Thread 12 in 16 - Hello World
Thread 11 in 16 - Hello World
Thread 10 in 16 - Hello World
Thread 9 in 16 - Hello World
Thread 8 in 16 - Hello World
Thread 7 in 16 - Hello World
Thread 6 in 16 - Hello World
Thread 5 in 16 - Hello World
Thread 4 in 16 - Hello World
Thread 3 in 16 - Hello World
Thread 2 in 16 - Hello World
Thread 1 in 16 - Hello World
Thread 0 in 16 - Hello World
Для продолжения нажмите любую клавишу . . .

Способ 2. Массив сообщений + вывод из главного потока

Идея: каждый поток записывает своё сообщение в свою ячейку массива (по индексу `thread_id`). После завершения параллельной области главный поток выводит массив в обратном порядке.

В данном случае отсутствует синхронизация внутри параллельной области, обеспечивается высокая надежность, но нужно заранее знать максимальное количество потоков



```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>
#include <iostream>

int main() {
    int quan;
    std::cin >> quan;
    omp_set_num_threads(quan);

    char messages[100][100];

    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        sprintf(messages[thread_id], "Thread %d in %d - Hello World\n",
            thread_id, quan);
    }

    for (int i = quan - 1; i >= 0; i--) {
        printf("%s", messages[i]);
    }

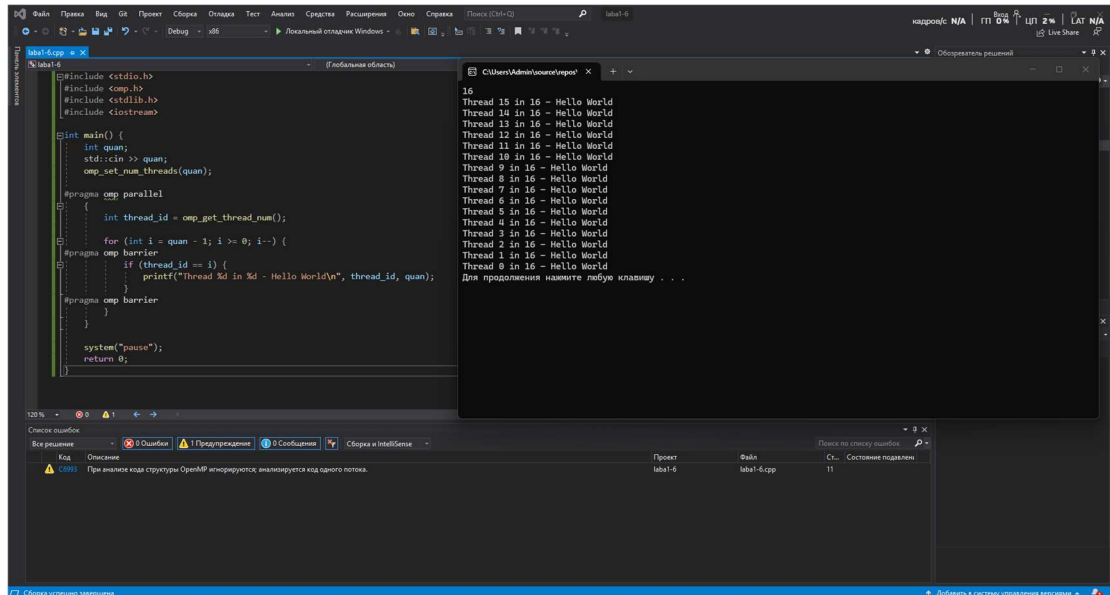
    system("pause");
    return 0;
}
```

16
Thread 15 in 16 - Hello World
Thread 14 in 16 - Hello World
Thread 13 in 16 - Hello World
Thread 12 in 16 - Hello World
Thread 11 in 16 - Hello World
Thread 10 in 16 - Hello World
Thread 9 in 16 - Hello World
Thread 8 in 16 - Hello World
Thread 7 in 16 - Hello World
Thread 6 in 16 - Hello World
Thread 5 in 16 - Hello World
Thread 4 in 16 - Hello World
Thread 3 in 16 - Hello World
Thread 2 in 16 - Hello World
Thread 1 in 16 - Hello World
Thread 0 in 16 - Hello World
Для продолжения нажмите любую клавишу . . .

Способ 3. Барьеры и обратный отсчет

Идея: использовать `#pragma omp barrier` для синхронизации. В цикле от `quan-1` до `0` на каждой итерации выводит только тот поток, чей ID совпадает с текущим значением счётчика. Остальные ждут на барьере

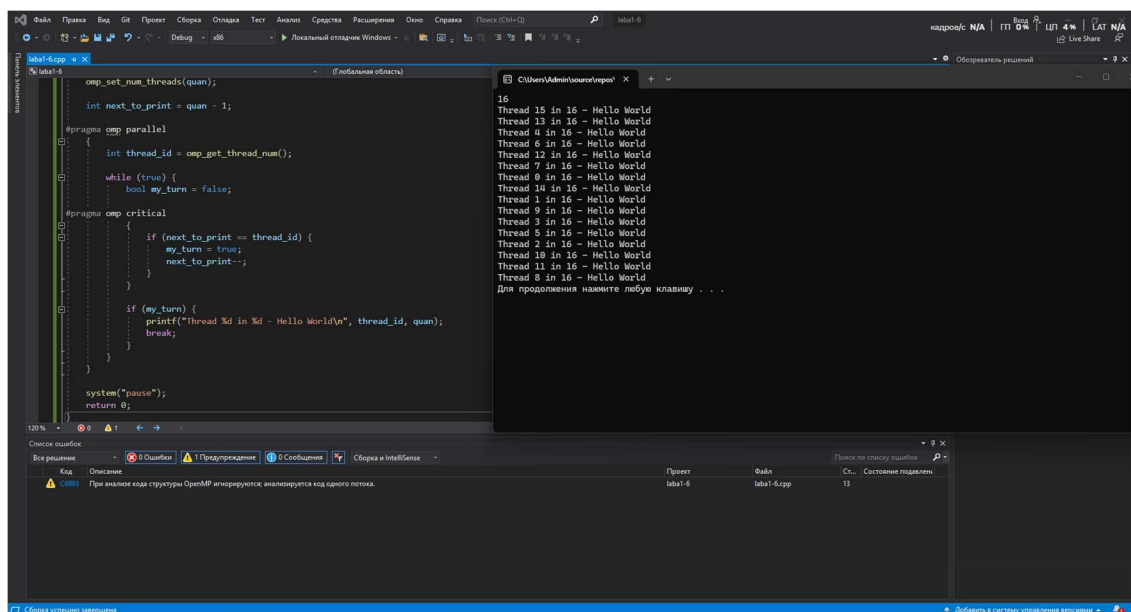
Много барьеров, следовательно, накладные расходы



Способ 4. Критическая секция и глобальные счетчик

Идея: завести глобальную переменную `next_to_print`, изначально равную `quan-1`. Каждый поток в цикле проверяет в критической секции, его ли очередь. Если да — выводит и уменьшает счётчик.

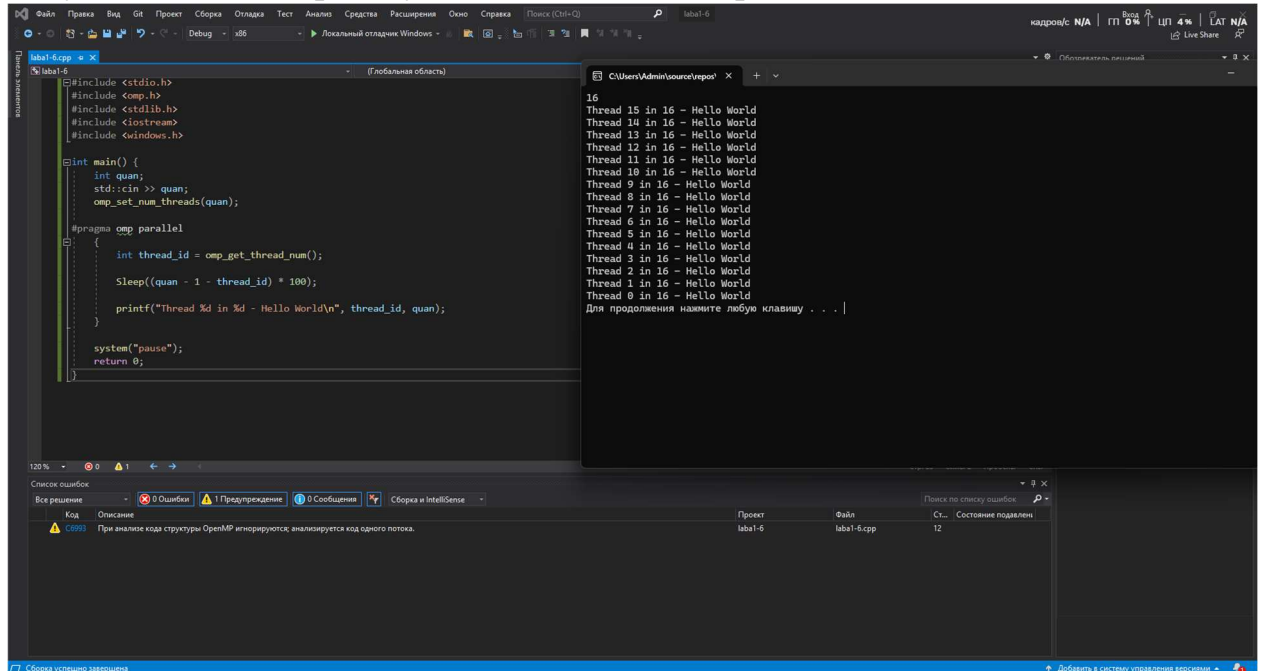
Потоки тратят мощность процессора впустую



Способ 5. Задержка Sleep, пропорциональная ID

Идея: заставить поток с большим ID "спать" меньше, а с маленьким — больше. Тогда потоки с большими ID выведут сообщение раньше.

Ненадёжно — при большой загрузке системы порядок может сбиться; искусственная задержка увеличивает общее время.



```
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>
#include <iostream>
#include <windows.h>

int main() {
    int quan;
    std::cin >> quan;
    omp_set_num_threads(quan);

#pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        Sleep((quan - 1 - thread_id) * 100);

        printf("Thread %d in %d - Hello World\n", thread_id, quan);
    }

    system("pause");
    return 0;
}
```

```
16
Thread 15 in 16 - Hello World
Thread 14 in 16 - Hello World
Thread 13 in 16 - Hello World
Thread 12 in 16 - Hello World
Thread 11 in 16 - Hello World
Thread 10 in 16 - Hello World
Thread 9 in 16 - Hello World
Thread 8 in 16 - Hello World
Thread 7 in 16 - Hello World
Thread 6 in 16 - Hello World
Thread 5 in 16 - Hello World
Thread 4 in 16 - Hello World
Thread 3 in 16 - Hello World
Thread 2 in 16 - Hello World
Thread 1 in 16 - Hello World
Thread 0 in 16 - Hello World
Для продолжения нажмите любую клавишу . . .
```

Код	Описание	Проект	Файл	Ст...	Состояние
C993	При анализе кода структуры OpenMP игнорируются; анализируется код одного потока.	label-6	label-6.cpp	12	Состояние подвешено